# 5_PRACTICAL_IMPLEMENTATION

# 💻 PRACTICAL IMPLEMENTATION GUIDE

## Hands-On Machine Learning Code Examples

---

## 🛠️ STEP-BY-STEP IMPLEMENTATION

### Environment Setup

*Bash*

```bash
# Required packages
pip install pandas scikit-learn matplotlib seaborn imbalanced-learn xgboost flask

# For advanced visualization
pip install plotly dash shap
```

### Complete Working Example

#### 1. Data Loading and Initial Exploration

*Python*

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import warnings
warnings.filterwarnings('ignore')

# Load dataset
df = pd.read_csv('dataset/mental_health_dataset.csv')
print(f"Dataset shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
```

```python

# Quick overview
print("\n=== DATASET OVERVIEW ===")
print(df.head())
print(f"\nMissing values: {df.isnull().sum().sum()}")
```

## 2. Comprehensive Preprocessing Pipeline

Python

```python
def preprocess_mental_health_data(df):
    """
    Complete preprocessing pipeline for mental health dataset
    """
    # Create copy to avoid modifying original
    data = df.copy()

    # 1. Feature Engineering
    print("🔧 Engineering features...")

    # Composite risk score
    data['risk_composite'] = (data['depression_score'] * 0.6) + (data['anxiety_score'] * 0.4)

    # Age groups
    data['age_group'] = pd.cut(data['age'], bins=[18, 30, 50, 65],
                               labels=['Young', 'Middle', 'Senior'])

    # Stress-sleep interaction
    data['stress_sleep_ratio'] = data['stress_level'] / data['sleep_hours']

    # Productivity efficiency
    data['efficiency_ratio'] = data['productivity_score'] / data['sleep_hours']

    # 2. Encode categorical variables
    print("🔢 Encoding categorical variables...")

    # Binary variables
    binary_mappings = {'Yes': 1, 'No': 0}
    data['mental_health_history'] = data['mental_health_history'].map(binary_mappings)
    data['seeks_treatment'] = data['seeks_treatment'].map(binary_mappings)

    # Multi-class encoding
```

```python
    le_gender = LabelEncoder()
    data['gender_encoded'] = le_gender.fit_transform(data['gender'])

    # One-hot encoding for employment and work environment
    data = pd.get_dummies(data, columns=['employment_status', 'work_environment'],
                          prefix=['emp', 'work'])

    # 3. Handle target variable
    print("🎯 Preparing target variable...")
    le_target = LabelEncoder()
    data['mental_health_risk_encoded'] = le_target.fit_transform(data['mental_health_risk'])

    return data, le_target

# Apply preprocessing
processed_df, label_encoder = preprocess_mental_health_data(df)
print(f"Processed dataset shape: {processed_df.shape}")
```

## 3. Feature Selection and Model Training

Python

```python
def train_risk_prediction_model(df):
    """
    Train mental health risk prediction model
    """
    # Select features for modeling
    feature_columns = [
        'age', 'gender_encoded', 'mental_health_history', 'seeks_treatment',
        'stress_level', 'sleep_hours', 'physical_activity_days',
        'depression_score', 'anxiety_score', 'social_support_score',
        'productivity_score', 'risk_composite', 'stress_sleep_ratio',
        'emp_Employed', 'emp_Self-employed', 'emp_Student',
        'work_Hybrid', 'work_On-site'
    ]

    X = df[feature_columns]
    y = df['mental_health_risk_encoded']

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

```

```
    # Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Train Random Forest
    print("🎯 Training Random Forest model...")
    rf_model = RandomForestClassifier(
        n_estimators=100,
        max_depth=12,
        min_samples_split=5,
        min_samples_leaf=2,
        random_state=42
    )

    rf_model.fit(X_train_scaled, y_train)

    # Evaluate model
    y_pred = rf_model.predict(X_test_scaled)
    y_pred_proba = rf_model.predict_proba(X_test_scaled)

    print("📈 MODEL PERFORMANCE:")
    print(classification_report(y_test, y_pred,
                                target_names=label_encoder.classes_))
    print(f"AUC-ROC Score: {roc_auc_score(y_test, y_pred_proba,
multi_class='ovr'):.3f}")

    # Feature importance
    feature_importance = pd.DataFrame({
        'feature': feature_columns,
        'importance': rf_model.feature_importances_
    }).sort_values('importance', ascending=False)

    print("\n⭐ TOP 10 MOST IMPORTANT FEATURES:")
    print(feature_importance.head(10))

    return rf_model, scaler, feature_columns

# Train the model
model, scaler, features = train_risk_prediction_model(processed_df)
```

---

## 🔮 PREDICTION FUNCTION WITH REAL EXAMPLES

### Complete Prediction System

Python

```python
def predict_mental_health_risk(model, scaler, feature_columns, input_data, label_encoder):
    """
    Make predictions for new mental health cases
    """
    # Convert input to DataFrame
    if isinstance(input_data, dict):
        input_df = pd.DataFrame([input_data])
    else:
        input_df = pd.DataFrame(input_data)

    # Apply same preprocessing as training data
    input_processed = input_df.copy()

    # Feature engineering (same as training)
    input_processed['risk_composite'] = (
        input_processed['depression_score'] * 0.6 +
        input_processed['anxiety_score'] * 0.4
    )
    input_processed['stress_sleep_ratio'] = (
        input_processed['stress_level'] / input_processed['sleep_hours']
    )

    # Encode categorical variables
    binary_mappings = {'Yes': 1, 'No': 0}
    input_processed['mental_health_history'] = input_processed['mental_health_history'].map(binary_mappings)
    input_processed['seeks_treatment'] = input_processed['seeks_treatment'].map(binary_mappings)

    # Gender encoding (assuming same encoder was saved)
    gender_mapping = {'Male': 0, 'Female': 1, 'Non-binary': 2, 'Prefer not to say': 3}
    input_processed['gender_encoded'] = input_processed['gender'].map(gender_mapping)

    # One-hot encoding
    employment_dummies = pd.get_dummies(input_processed['employment_status'],
                                        prefix='emp')
    work_dummies = pd.get_dummies(input_processed['work_environment'],
                                  prefix='work')

    # Combine all features
    input_final = pd.concat([
        input_processed[['age', 'gender_encoded', 'mental_health_history',
                         'seeks_treatment', 'stress_level', 'sleep_hours',
```

```
                        'physical_activity_days', 'depression_score',
                        'anxiety_score', 'social_support_score',
                        'productivity_score', 'risk_composite',
    'stress_sleep_ratio']],
            employment_dummies,
            work_dummies
        ], axis=1)

        # Ensure all required columns are present
        for col in feature_columns:
            if col not in input_final.columns:
                input_final[col] = 0

        # Select and order features
        X_input = input_final[feature_columns]

        # Scale features
        X_input_scaled = scaler.transform(X_input)

        # Make prediction
        prediction = model.predict(X_input_scaled)
        probabilities = model.predict_proba(X_input_scaled)

        # Format results
        risk_levels = label_encoder.inverse_transform(prediction)
        confidence = np.max(probabilities, axis=1)

        return {
            'predicted_risk': risk_levels[0],
            'confidence': f"{confidence[0]:.1%}",
            'probabilities': {
                'Low': f"{probabilities[0][0]:.1%}",
                'Medium': f"{probabilities[0][1]:.1%}",
                'High': f"{probabilities[0][2]:.1%}"
            }
        }

    # Example usage with detailed scenarios
```

---

# 🎯 THREE DETAILED PREDICTION EXAMPLES

## Example 1: High-Risk Young Professional

Python

```python
# Input data for a struggling young professional
high_risk_case = {
    'age': 28,
    'gender': 'Female',
    'employment_status': 'Employed',
    'work_environment': 'On-site',
    'mental_health_history': 'Yes',
    'seeks_treatment': 'No',
    'stress_level': 9,
    'sleep_hours': 4.5,
    'physical_activity_days': 1,
    'depression_score': 26,
    'anxiety_score': 18,
    'social_support_score': 35,
    'productivity_score': 55
}

result1 = predict_mental_health_risk(model, scaler, features, high_risk_case, label_encoder)
print("=== EXAMPLE 1: HIGH-RISK YOUNG PROFESSIONAL ===")
print(f"Predicted Risk: {result1['predicted_risk']}")
print(f"Confidence: {result1['confidence']}")
print("Probability Breakdown:")
for risk, prob in result1['probabilities'].items():
    print(f"  {risk}: {prob}")

# Explanation of prediction logic
print("\n🔍 LOGICAL REASONING:")
print("- Extremely high depression (26) and anxiety (18) scores")
print("- Severe sleep deprivation (4.5 hours) amplifies risk")
print("- High stress (9/10) without treatment history indicates urgency")
print("- Low social support (35) removes protective buffering")
print("- Declined productivity (55) suggests functional impairment")
```

## Example 2: Moderate-Risk Student

*Python*

```python
# Input data for a stressed college student
student_case = {
    'age': 22,
    'gender': 'Male',
    'employment_status': 'Student',
    'work_environment': 'Remote',
    'mental_health_history': 'No',
    'seeks_treatment': 'No',
    'stress_level': 7,
```

```
    'sleep_hours': 6.2,
    'physical_activity_days': 3,
    'depression_score': 15,
    'anxiety_score': 12,
    'social_support_score': 65,
    'productivity_score': 72
}

result2 = predict_mental_health_risk(model, scaler, features,
student_case, label_encoder)
print("\n=== EXAMPLE 2: MODERATE-RISK STUDENT ===")
print(f"Predicted Risk: {result2['predicted_risk']}")
print(f"Confidence: {result2['confidence']}")
print("Probability Breakdown:")
for risk, prob in result2['probabilities'].items():
    print(f"  {risk}: {prob}")

# Explanation of prediction logic
print("\n🔍 LOGICAL REASONING:")
print("- Moderate depression (15) and anxiety (12) levels")
print("- Academic stress typical for student population")
print("- Adequate sleep (6.2 hours) provides some protection")
print("- Good social support (65) acts as buffer")
print("- Reasonable productivity (72) indicates functional capacity")
print("- Male students typically show delayed help-seeking")
```

## Example 3: Low-Risk Senior Professional

Python

```
# Input data for a well-adjusted senior professional
senior_case = {
    'age': 55,
    'gender': 'Female',
    'employment_status': 'Employed',
    'work_environment': 'Hybrid',
    'mental_health_history': 'No',
    'seeks_treatment': 'No',
    'stress_level': 3,
    'sleep_hours': 7.8,
    'physical_activity_days': 5,
    'depression_score': 4,
    'anxiety_score': 2,
    'social_support_score': 88,
    'productivity_score': 94
}

```

```python
result3 = predict_mental_health_risk(model, scaler, features, senior_case, label_encoder)
print("\n=== EXAMPLE 3: LOW-RISK SENIOR PROFESSIONAL ===")
print(f"Predicted Risk: {result3['predicted_risk']}")
print(f"Confidence: {result3['confidence']}")
print("Probability Breakdown:")
for risk, prob in result3['probabilities'].items():
    print(f"  {risk}: {prob}")

# Explanation of prediction logic
print("\n🔍 LOGICAL REASONING:")
print("- Minimal depression (4) and anxiety (2) symptoms")
print("- Excellent sleep quality (7.8 hours)")
print("- Low stress levels (3/10) indicate good coping")
print("- Strong social support (88) provides robust protection")
print("- High productivity (94) demonstrates optimal functioning")
print("- Hybrid work environment offers flexibility benefits")
print("- Age-related wisdom and coping strategies likely developed")
```

---

# 📊 MODEL EXPLANATION & INTERPRETABILITY

## SHAP Values for Interpretability

Python

```python
import shap

def explain_prediction(model, scaler, feature_columns, input_data):
    """
    Explain individual predictions using SHAP values
    """
    # Prepare input data (same preprocessing as before)
    # ... (preprocessing code here)

    # Calculate SHAP values
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_input_scaled)

    # Feature contribution analysis
    feature_contributions = pd.DataFrame({
        'feature': feature_columns,
        'contribution': shap_values[0]  # For first prediction
    }).sort_values('contribution', key=abs, ascending=False)

    return feature_contributions
```

```python

# Example explanation for high-risk case
contributions = explain_prediction(model, scaler, features, high_risk_case)
print("\n📊 FEATURE CONTRIBUTION ANALYSIS (HIGH-RISK CASE):")
print(contributions.head(8))
```

---

## 🚀 PRODUCTION DEPLOYMENT TEMPLATE

### Flask API Implementation

Python

```python
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)

# Load trained model and preprocessors
model = joblib.load('mental_health_model.pkl')
scaler = joblib.load('feature_scaler.pkl')
label_encoder = joblib.load('label_encoder.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Get input data
        input_data = request.json

        # Make prediction
        result = predict_mental_health_risk(
            model, scaler, features, input_data, label_encoder
        )

        # Add recommendations based on prediction
        recommendations = generate_recommendations(result['predicted_risk'])
        result['recommendations'] = recommendations

        return jsonify(result)

    except Exception as e:
        return jsonify({'error': str(e)}), 400

def generate_recommendations(risk_level):
```

```python
    """Generate personalized recommendations"""
    recommendations = {
        'High': [
            'Immediate professional consultation recommended',
            'Consider employee assistance program',
            'Emergency contact information provided',
            'Flexible work arrangements suggested'
        ],
        'Medium': [
            'Schedule mental health screening',
            'Explore stress management resources',
            'Consider peer support groups',
            'Review work-life balance'
        ],
        'Low': [
            'Maintain current healthy habits',
            'Regular mental health check-ins',
            'Continue strong social connections',
            'Preventive wellness activities'
        ]
    }
    return recommendations.get(risk_level, ['Consult healthcare provider'])

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

---

## 📈 BATCH PROCESSING FOR ORGANIZATIONS

### Corporate Mental Health Screening

*Python*

```python
def batch_process_employees(employee_data_file):
    """
    Process large employee datasets for organizational screening
    """
    # Load employee data
    employees = pd.read_csv(employee_data_file)

    # Process each employee
    results = []
    for idx, employee in employees.iterrows():
        prediction = predict_mental_health_risk(
            model, scaler, features, employee.to_dict(), label_encoder
```

```
            )

            # Add employee identifier
            prediction['employee_id'] = employee.get('employee_id', idx)
            results.append(prediction)

        # Create summary report
        results_df = pd.DataFrame(results)

        # Risk distribution
        risk_summary = results_df['predicted_risk'].value_counts()
        print("📊 ORGANIZATIONAL RISK DISTRIBUTION:")
        print(risk_summary)

        # Priority recommendations
        high_risk_employees = results_df[results_df['predicted_risk'] == 'High']
        print(f"\n🚨 HIGH-RISK EMPLOYEES REQUIRING IMMEDIATE ATTENTION: {len(high_risk_employees)}")

        return results_df

# Example usage
# employee_results = batch_process_employees('company_employees.csv')
```

---

*This implementation guide provides production-ready code for mental health risk prediction with comprehensive examples and deployment strategies*